

FreshFind Website Project Report

Requirements, design, test data and installation

Version 1.0 | 25 September 2026

FreshFind is a browser based guide to selected Lagos farmers markets and seasonal produce. This report records the features present in the current React application, explains how its static data and rule based assistant work, and compares the implementation with the supplied FreshFind software requirements specification. It also gives test scenarios and installation instructions for review or handover.

Document basis	Role in this report
FreshFind Web Innovation Unleashed SRS, version 1.0	Requirements baseline and required report sections
FreshFind repository inspected 25 September 2026	Evidence for current features, structure and test data
CarBreezy project report example	General report structure and presentation reference only

Contents

- 1 Problem Definition and Project Scope
- 2 Implemented Website Features
- 3 Design Specifications and Architecture
- 4 User Flows and Data Flow
- 5 Data Model and Test Data
- 6 Verification and Quality
- 7 Installation and Operation
- 8 Requirements Remaining and Handover

1 Problem Definition and Project Scope

Residents often learn about local market days, hours and available produce through scattered posts or word of mouth. FreshFind brings a selected set of Lagos market details into one website so a visitor can discover a market, check its regular schedule and explore produce before planning a trip. The current dataset is illustrative, so the site advises visitors to confirm schedules directly with a market.

The application is a single page React website. It runs in the browser, reads bundled JSON data, and has no application server or live market database. Market and produce listings do not update from the website itself. Saved items are kept in the visitor's browser storage.

Project objectives

- Help visitors search markets by name, area or listed produce.
- Show regular opening days and hours, market locations and typical items.
- Give visitors a searchable guide to produce and recorded seasons.
- Provide a rule based assistant for common market and produce questions.
- Let visitors save markets and produce for later on the same browser.

2 Implemented Website Features

Area	Current behaviour
Home	Tree themed landing page with canopy, three scroll driven branches for markets, produce and saved items, roots and soil footer. Search opens the market directory.
Market directory	Lists five markets. Free text searches name, area, address and listed produce. Area and open today controls filter the list.
Market detail	Shows description, regular days and hours, produce, current open status and a Google Maps directions link.
Produce guide	Lists eight produce entries. Search, category and current season controls narrow results.
Saved items	Visitors save or remove markets and produce. The collection persists in browser local storage.
Assistant	Floating widget accepts typed questions and quick prompts; responses come from local JSON and fixed rules.

Current website screens

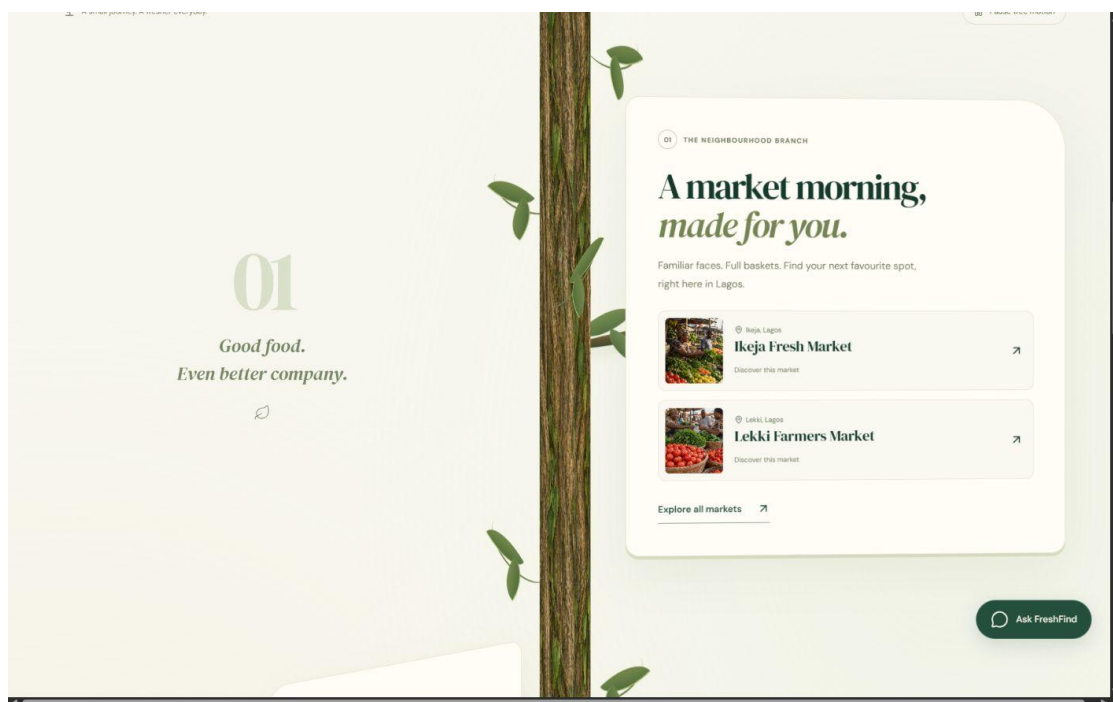


Figure 1 Home page market branch attached to the tree trunk

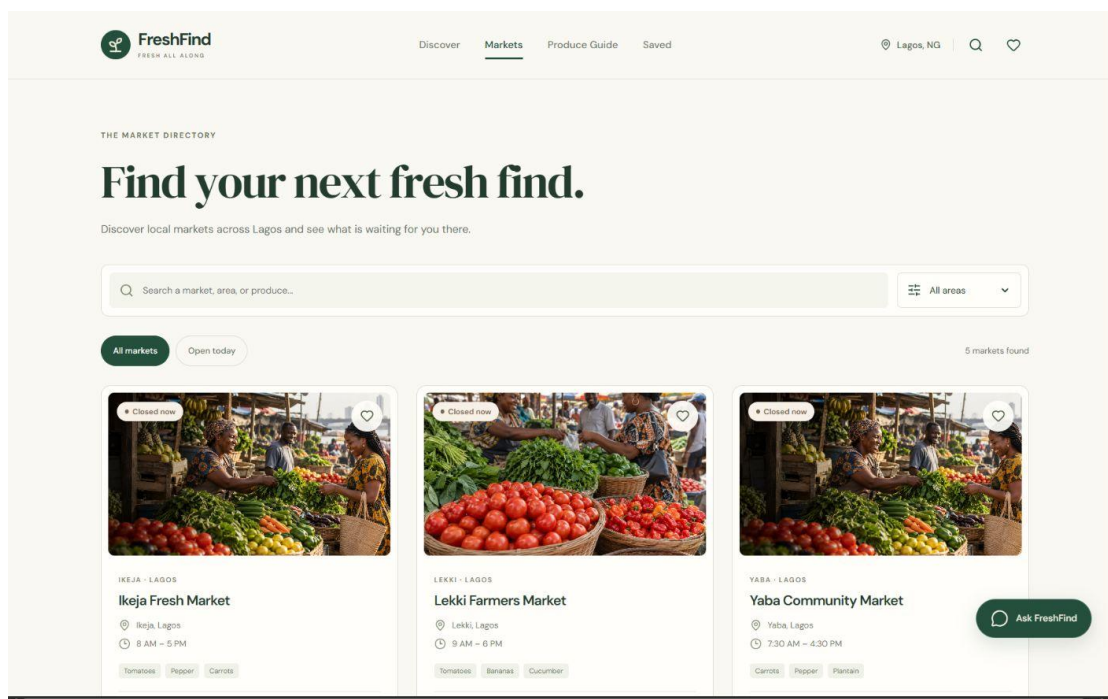


Figure 2 Searchable Lagos market directory

3 Design Specifications and Architecture

React 19 renders the interface and React Router maps browser paths to pages. Vite 8 builds the project. CSS handles the responsive layout and tree animation; Lucide supplies icons. The home page rotates its decorative trunk and section panels from scroll position. Animation can be paused on desktop and is disabled for reduced motion or narrow screens. The earlier home concepts remain available in the repository for reference.

Site map

Path	Purpose
/	Tree themed landing page
/markets	Market search and filters
/markets/:id	Individual market details
/produce-guide	Produce catalogue
/saved	Visitor's saved collection
/h and /h2	Earlier home page prototypes retained for comparison

Application components

Layer	Responsibility
Page and navigation	App routes, Navbar and Footer display the active page and shared shell.
Feature components	MarketCard, ProduceCard and ChatBot render reusable visitor actions.
Local data	JSON files supply markets, produce, seasonal entries and assistant responses.
Browser state	SavedContext reads and writes localStorage key freshfind-saved.
External navigation	Market details open Google Maps search links for coordinates; this is not an embedded map.

4 User Flows and Data Flow

Market discovery flow

Step	Visitor action	Website response
1	Open the home page or market directory	Display the tree introduction or all market cards.

2	Enter a market, area or produce term	Filter local market data and update the result count.
3	Choose an area or open today	Narrow the cards using area and Lagos market day rules.
4	Open a market card	Show schedule, listed produce, location and directions link.
5	Save the market	Add its ID to localStorage and show it under Saved.



Figure 3 Market discovery flowchart

Level 0 data flow

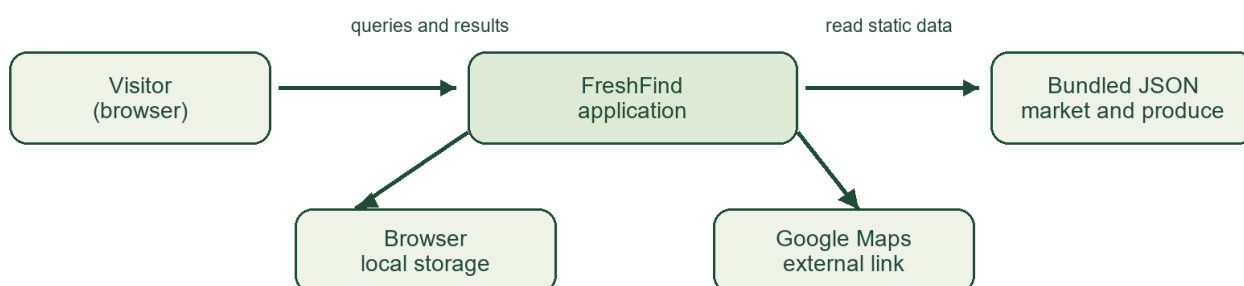


Figure 4 Level 0 data flow diagram

The application matches the request against static data, renders cards or assistant replies, and returns results to the visitor. A save action writes only to that visitor's browser localStorage. A directions action opens Google Maps in a new browser tab. No information is sent to a FreshFind server.

Assistant decision flow

Question or quick prompt > text normalization > produce, market, area, day or season rule > matching records and reply card > fallback reply if no rule matches

The assistant is rule based and does not connect to an external AI service. Its open today and season rules use the browser's local date; market card open status uses the Africa/Lagos time zone. This difference matters when a visitor browses from another time zone.

5 Data Model and Test Data

The project stores five example Lagos markets and eight produce entries in prepopulated JSON files. The market dataset contains ID, name, area, address, description, regular days, opening and closing times, typical produce and coordinates. A produce record contains ID, name, category, description and named season months. Assistant responses and seasonal prompts have separate local JSON files.

Dataset	Records used	Sample values
Markets	5	Ikeja, Lekki, Yaba, Surulere, Victoria Island
Produce	8	Tomatoes, Carrots, Spinach, Bananas, Yam, Pepper, Cucumber, Watermelon
Saved IDs	Per browser	Two arrays named markets and produce in localStorage

Representative test scenarios

Scenario	Input	Expected result
Market search	Ikeja	Ikeja Fresh Market appears.
Produce search in markets	Tomatoes	Markets listing Tomatoes remain.

Area filter	Lekki	Only the Lekki market card appears.
Produce category	Fruits	Bananas and Watermelon remain.
Bookmark	Save Ikeja market	Card is marked saved and appears under Saved markets after reload.
Assistant	Where can I find tomatoes?	Matching markets appear as linked reply cards.
Empty results	Unlisted search term	A no results message is shown.

6 Verification and Quality

The project completed a production Vite build and ESLint check during this documentation pass. Browser inspection confirmed the home page, market branch and market directory render with expected navigation and content at a desktop viewport. These checks are limited to the inspected environment; a full cross browser, accessibility and Lighthouse audit is still required before formal acceptance.

Check	Observed result	Further validation
npm run build	Passed	Repeat after data or UI changes.
npm run lint	Passed	Repeat after code changes.
Desktop browser inspection	Home and directory rendered	Test all paths and controls on mobile, keyboard and other browsers.
Lighthouse	Not run	Record performance, accessibility and SEO scores.

Accessibility provisions visible in the implementation include labelled search inputs, a skip link, keyboard focus styles, descriptive buttons, reduced motion handling and alternate text for content images. The chat widget returns focus to its launcher when closed. These provisions should be checked with keyboard and screen reader testing rather than treated as certification.

7 Installation and Operation

Prerequisites: a recent Node.js release compatible with Vite 8, npm and a current browser. The repository contains a package lock file for repeatable dependency installation. No database, API key or backend service is required for the current website.

Step	Command or action	Result
1	Open the FreshFind repository in a terminal	The project folder is the working directory.
2	npm ci	Install dependencies from package lock.
3	npm run dev	Start the Vite local development server; open the URL it prints.
4	npm run build	Create a production build in dist.
5	npm run preview	Serve that build locally for review.
6	npm run lint	Run the repository's ESLint check.

For static hosting, publish the contents of dist and configure the host to serve index.html for client side routes such as /markets/1. Market data lives in src/JSON; edit those files, check references between market produce names and guide entries, then rebuild. Browser bookmarks are local to each visitor and will not follow them to another device.

8 Requirements Remaining and Handover

The SRS is broader than the current implementation. The following items should be completed or explicitly waived before calling the full specification delivered.

SRS item	Current state	Work remaining
Directory day and sort controls	Partial	Add a chosen weekday filter and alphabetic, proximity or next open sorting.
Embedded map	Partial	Market pages link to Google Maps but do not embed a map.
Bookmark notes, export and sharing	Missing	Add notes and export or share actions if required.
About and Contact pages	Missing from active routes	Build and link both pages, or obtain a scope change.
Visitor counter, clock and geolocation	Missing from active UI	Implement only if retained as acceptance criteria.

Login and signup mock controls	Missing	Add the requested design only controls if required.
Lighthouse and full browser testing	Not recorded	Run and document the acceptance checks.
Demonstration video and source ZIP	Not part of website code	Prepare the SRS submission artifacts separately.

Source references

FreshFind Web Innovation Unleashed Software Requirements Specification, version 1.0, supplied PDF. FreshFind source repository and bundled JSON data inspected 25 September 2026. CarBreezy project report supplied as an example of report organization; its project claims and code were not used as FreshFind evidence.